

EDS Lab Assignments

Practical 1

Name: Dhanashri Mamidwar

PRN: 202501040059

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m \times v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.1. Calculate Momentum' is visible, including the formula $p = m \times v$ and the input/output requirements. The main editor shows the following Python code:

```
1 mass=float(input())
2 velocity=float(input())
3 momentum=mass*velocity
4 print(f"{momentum:.2f}kgm/s")
```

Below the code, the test results are shown. Two test cases have passed:

Test Case	Expected output	Actual output
Test case 1	5.0 10.0 10.00kg/s	5.0 10.0 10.00kg/s
Test case 2	2.0 4.0 11.00kg/s	2.0 4.0 11.00kg/s

The interface also shows performance metrics: Average time 0.009 s, Maximum time 0.027 s, and a status of '2 out of 2 shown test case(s) passed'.

1.1.2 Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays the CodedTANTRA IDE interface. On the left, the problem statement and constraints are visible. The main editor shows the Python code for conditional calculation based on the number of digits. The right sidebar shows the test results, indicating that 4 out of 4 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases are listed with their expected and actual outputs.

Test Case	Expected Output	Actual Output
Test case 1	81	81
Test case 2		
Test case 3		
Test case 4		

1.1.3 Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays a coding platform interface for the problem "1.1.3. Days Between Two Dates". The left sidebar contains the problem description, input/output formats, and sample test cases. The main area shows a Python code editor with the solution code. The right sidebar displays the execution results, including a terminal window and a test cases summary.

Problem Statement: Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD
- The second line contains the second date in the format YYYY-MM-DD

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

Test case	Input 1	Input 2	Output
Test case 1	2014-07-01	2014-11-01	123
Test case 2	2014-07-01	2014-07-11	9

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Execution Results:

- Average time: 0.007 s, Maximum time: 0.014 s
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1:

Expected output	Actual output
2014-07-01	2014-07-01
2014-11-01	2014-11-01
123	123

Test Case 2:

Expected output	Actual output
2014-07-01	2014-07-01
2014-07-11	2014-07-11
9	9

1.1.4 Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays a coding platform interface for a problem titled "1.1.4. Reverse a Number". The problem statement asks to write a Python program to reverse the digits of a number and print the reversed number. The input format is a single integer, and the output format is a single integer representing the reversed number. Sample test cases are provided: Test case 1 with input 5367 and output 7635, and Test case 2 with input 8132 and output 231. The user's code is shown in the editor, and the test results indicate that 2 out of 2 shown test cases passed and 3 out of 3 hidden test cases passed. The interface includes a terminal, test cases section, and navigation buttons at the bottom.

1.1.4. Reverse a Number 0.002 s 2.29 ms 0.003 s 3.68 ms Submit Debugger

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

Test Case	Input	Output
Test case 1	5367	7635
Test case 2	8132	231

Test Results:

- Average time: 0.002 s, Maximum time: 0.003 s
- 2 out of 2 shown test case(s) passed
- 3 out of 3 hidden test case(s) passed

Test case 1: Expected output: 5367, Actual output: 7635

Test case 2: Expected output: 8132, Actual output: 231

Code:

```
1 num = int(input())
2 reversed_num = int(str(num)[::-1])
3 print(reversed_num)
```

Terminal: Test cases < Prev Next >

1.1.5 Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.5. Multiplication Table' is visible, including the input/output format and sample test cases. The main editor shows the Python code:

```
n = int(input())  
for i in range(1,11):  
    print(n, "x", i, "=", n*i)
```

 The right sidebar shows the execution results. It indicates that the program passed 2 out of 2 shown test cases and 2 out of 2 hidden test cases. The output for the shown test cases is displayed as follows:

Expected output	Actual output
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40
5 x 9 = 45	5 x 9 = 45
5 x 10 = 50	5 x 10 = 50

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
```

```

aggregate_percentage, grade = result
print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
print(f"Grade: {grade}")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a problem statement titled "12.1. Pass or Fail" is shown. It asks for a Python program that calculates the aggregate percentage and grade based on marks in multiple courses. The program should accept the number of courses and the marks in each course as input. The grade is determined based on the aggregate percentage: First Division (75% and above), Second Division (60% to 74%), Third Division (40% to 59%), and Fail (below 40%).

The main editor shows the following Python code:

```

def cal(marks, total_courses):
    if any(mark < 40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = (total_marks / (total_courses * 100)) * 100
    if aggregate_percentage >= 75:
        grade = "First Division"
    elif aggregate_percentage >= 60:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    else:
        grade = "Fail"
    return (aggregate_percentage, grade)

num_courses = int(input())
marks = list(map(int, input().split()))
result = cal(marks, num_courses)
if result == "Fail":
    print("Fail")
else:
    aggregate_percentage, grade = result
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
    print(f"Grade: {grade}")

```

Below the code editor, the "Test cases" tab is active, showing the results of five test cases. The test cases are as follows:

Test Case	Input	Expected Output	Actual Output
Test case 1	5 55 65 78 80	Aggregate Percentage: 71.75 Grade: First Division	Aggregate Percentage: 71.75 Grade: First Division
Test case 2	5 56 78 97 86 93	Aggregate Percentage: 82.88 Grade: Distinction	Aggregate Percentage: 82.88 Grade: Distinction
Test case 3	5 55 65 42 57 34	Fail	Fail
Test case 4	5 55 54 53	Aggregate Percentage: 54.88 Grade: Second Division	Aggregate Percentage: 54.88 Grade: Second Division
Test case 5	5 56 65 62 68 62	Aggregate Percentage: 63.28 Grade: Third Division	Aggregate Percentage: 63.28 Grade: Third Division

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Output:

The screenshot displays a coding platform interface for the '1.2.2. Fibonacci Series' problem. The left sidebar contains the problem description, input/output formats, and sample test cases. The main area shows the Python code for the recursive Fibonacci function and its execution. The code is as follows:

```
def fibonacci(n):  
    if n==1:  
        return 0  
    if n==2:  
        return 1  
    return fibonacci(n-1)+fibonacci(n-2)  
  
n = int(input())  
for i in range(1, n+1):  
    print(fibonacci(i), end=" ")
```

The execution results show that the code passed all test cases. The performance metrics are: Average time: 0.008 s, Maximum time: 0.021 s. The test cases are:

Test Case	Expected Output	Actual Output
Test case 1	0 1 1 2 3	0 1 1 2 3
Test case 2	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

The interface also includes a 'Terminal' tab and navigation buttons at the bottom: < Prev, Reset, Submit, Next >.

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*",end = "")
    print()
```

Output:

The screenshot displays a coding platform interface for a problem titled "1.2.3. Pattern - 1". The problem statement asks for a Python program to print a right-angled triangle pattern of asterisks. The input format specifies an integer for the number of rows, and the output format specifies the pattern of asterisks. Sample test cases are provided for inputs 3 and 4. The code editor shows the following Python code:

```
1 n=int(input())
2 for i in range(1,n+1):
3     print('* ',end = '')
4     print()
```

The interface also shows a debugger window with the following output:

Average time: 0.002 s, Minimum time: 0.003 s, 2 out of 2 shown test case(s) passed, 4 out of 4 hidden test case(s) passed.

Test case 1: Expected output: 3, Actual output: 3. Test case 2: Expected output: 4, Actual output: 4.

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot displays the CODETANTRA online IDE interface. On the left, the problem statement for '1.2.4. Pattern - 2' is shown, including the input/output formats and sample test cases. The main area contains a code editor with the following Python code:

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()
```

Below the code editor, the test runner shows the results of two test cases. Test case 1 (n=5) and Test case 2 (n=8) both passed, with the expected output matching the actual output. The output for n=5 is:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

The output for n=8 is:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
```